

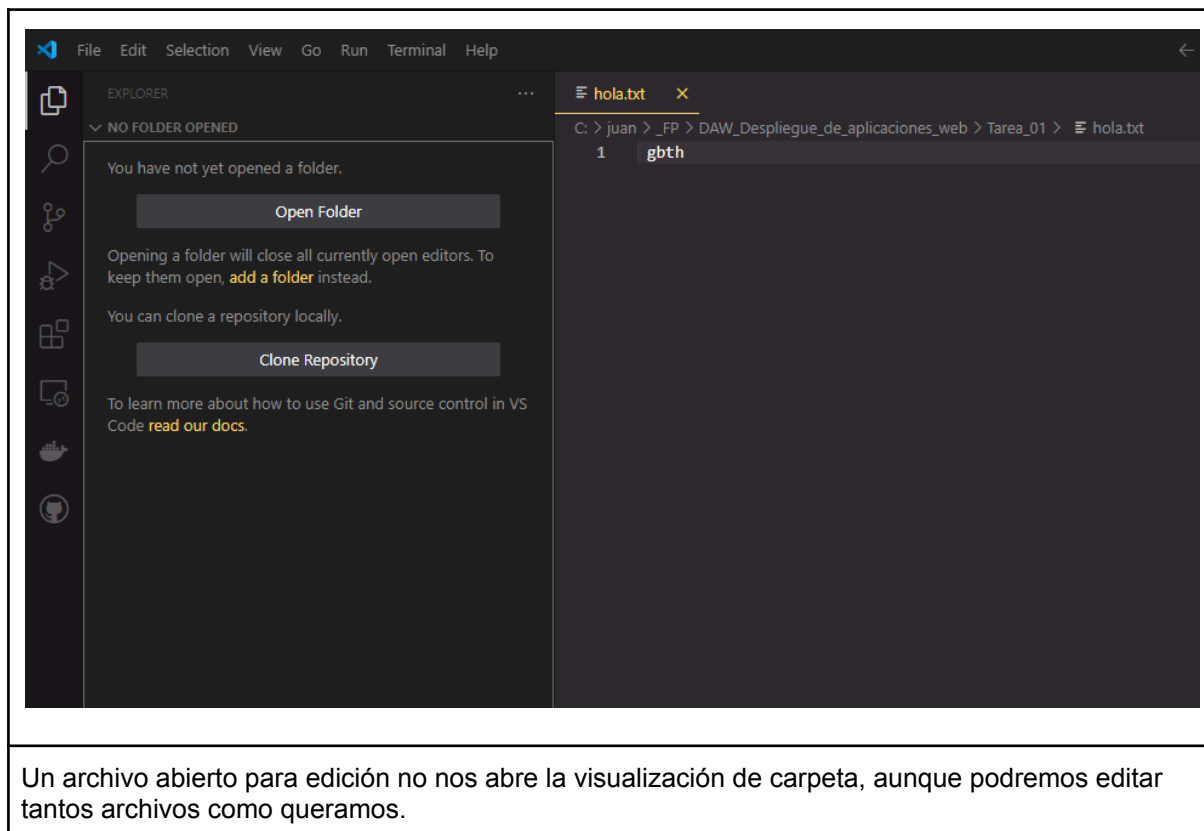
Desarrollo de aplicaciones Web en Entorno Cliente (DWECE). Entrega tareas unidad 01.

1. Gestor de espacios de proyectos y carpetas con VS Code.....	2
2. Gestor de versiones y copias de seguridad con GitHub, con ejemplos de PULL, COMMIT y PUSH.....	4
Commit y push.....	5
Pull.....	6
3. Mostrar en tiempo real los cambios en el HTML y CSS a medida que se hacen.....	7
4. Editor de texto con macros.....	8
5. Autocompletar con los métodos de las clases del API de JavaScript.....	10
6. Snippets HTML.....	10
7. Depuración con Chrome (Pestañas, Elements, Console y Sources).....	12
8. Depuración avanzada en Chrome: Simulación de parámetros típicos de los móviles en Chrome de escritorio: responsive, móvil, geolocalización.....	15

1. Gestor de espacios de proyectos y carpetas con VS Code

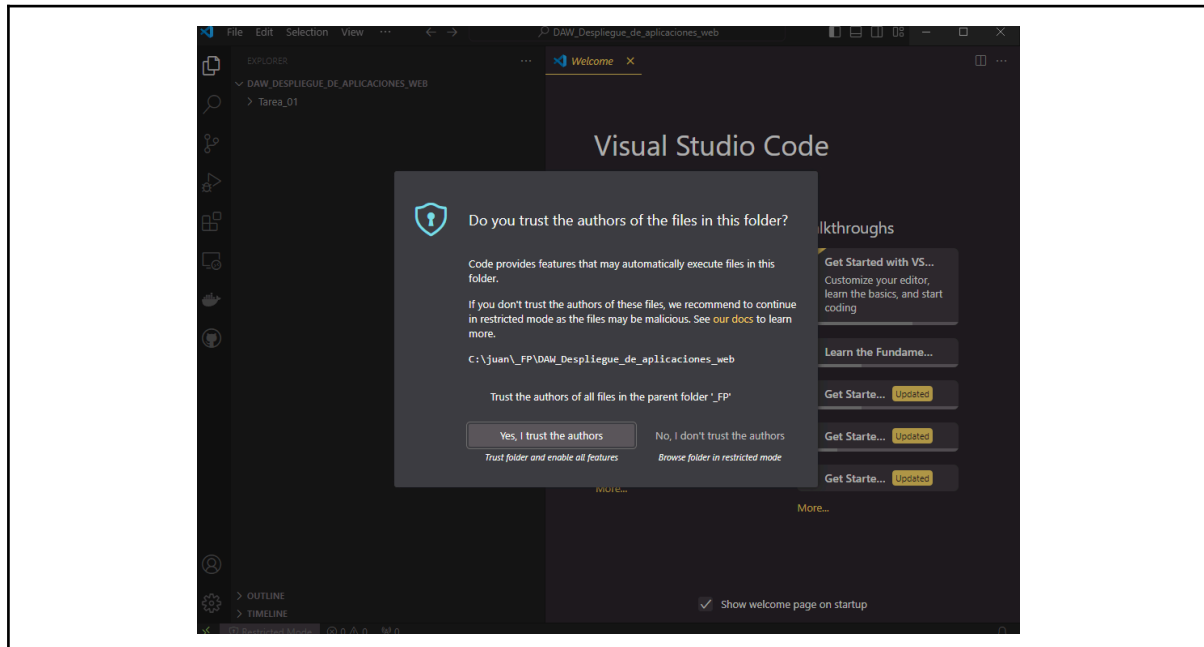
VS Code es un IDE (Integrated Development Environment, Entorno de desarrollo integrado) que puede actuar solo como editor de un archivo único, como editor de archivos ubicado en una carpeta (con explorador incluido) o como un entorno de desarrollo trabajando sobre una o varias carpetas y con configuraciones específicas para el tipo de proyecto que estamos desarrollando.

Si se abre un archivo nada más desde el explorador de archivos del sistema operativo o línea de comandos, no dispondremos de la vista de explorador.

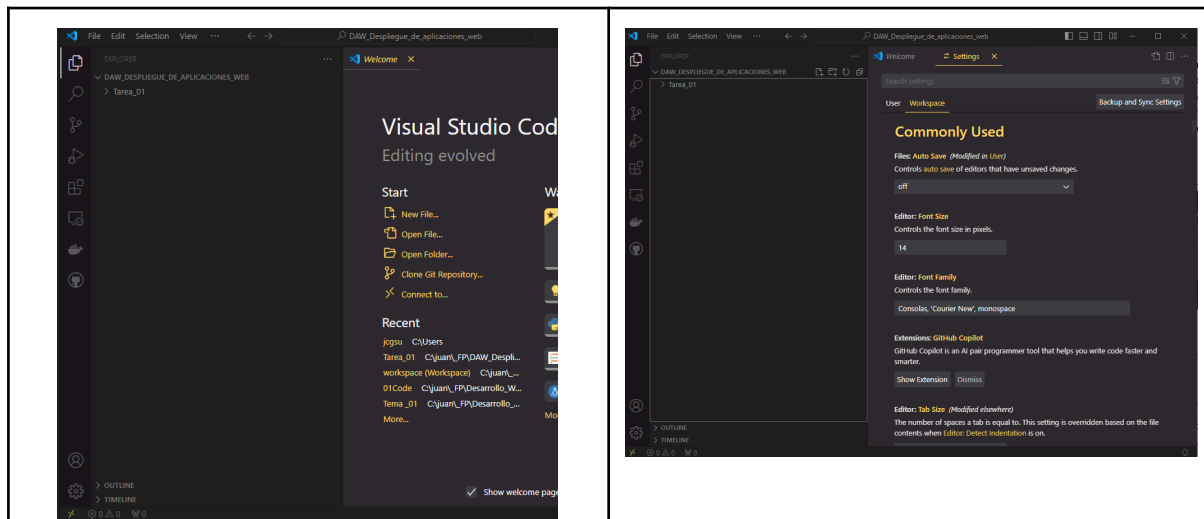


Sin embargo, si abrimos una carpeta con el menú contextual del explorador de archivos o desde la línea de comandos (`code .` para la carpeta actual en línea de comandos), **VS Code** se abrirá en modo de exploración de la carpeta, pidiéndonos en primer lugar que confirmemos que confiamos en ese contenido y viendo los contenidos de esa carpeta en el explorador de archivos incorporado. Además, en

modo carpetas podremos configurar aspectos específicos para esa carpeta a través del menú de configuración.



En el momento de abrir en modo carpeta, nos pide confirmación de confianza sobre sus contenidos.



Una vez aceptado podemos ver el explorador de archivos a la izquierda, por defecto.

Podremos configurar aspectos específicos para esta carpeta a través de la configuración del workspace tanto visualmente como editando el json correspondiente.

Por último está el concepto de workspace tradicional de los IDEs's de desarrollo de software.

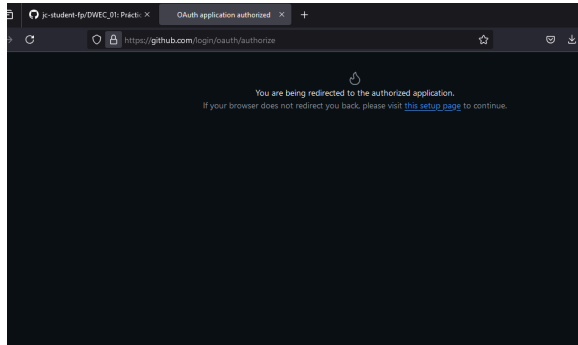
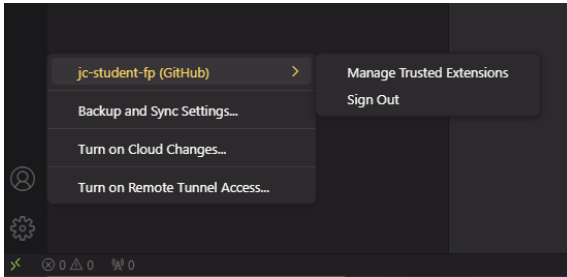
En **VS Code**, un **workspace o área de trabajo** puede ser de una sola carpeta o de varias (multi-root workspaces), esto último se consigue con la aplicación del fichero de configuración json con nombre del workspace que deseemos establecer y de extensión `.code-workspace`, este fichero en un ejemplo muy simple que configura un workspace de dos raíces, es de la forma:

```
{
  "folders": [
    {
      "path": "my-folder-a"
    },
    {
      "path": "my-folder-b"
    }
  ]
}
```

Ese fichero además de permitir el trabajo con espacios multi raíz, permite también la configuración específica de aspectos que solo queremos aplicar en ese proyecto, como se ha visto antes, y lo podremos crear y mantener para espacios de una sola carpeta en caso de que necesitemos configuraciones específicas.

2. Gestor de versiones y copias de seguridad con GitHub, con ejemplos de `PULL`, `COMMIT` y `PUSH`

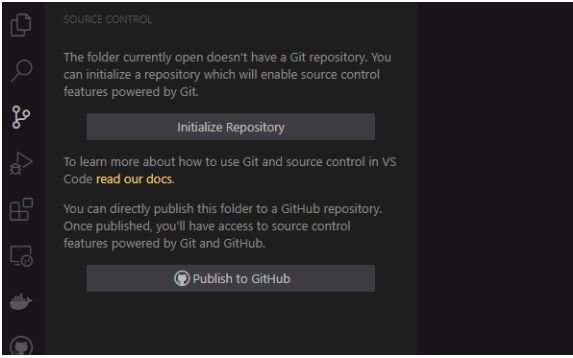
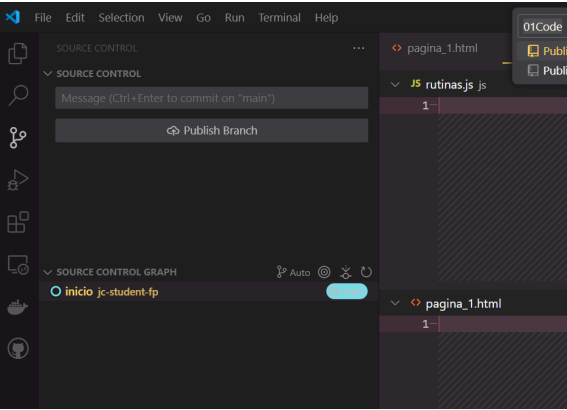
Una vez creado un nuevo usuario en GitHub con sus credenciales asociadas a mi email en fpdrioja, la configuración en VS Code comienza con la autenticación del usuario con las credenciales de github (que se hace por intermedio de la web).

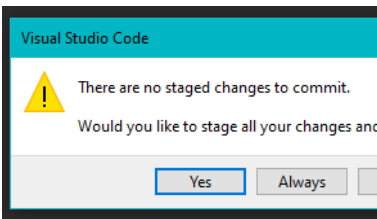
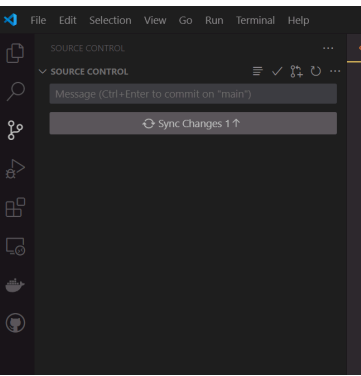
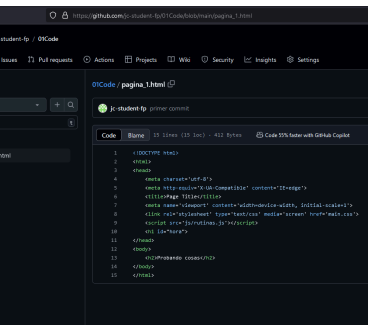
	
Autenticación en la web de GitHub a petición de VS Code.	Usuario de GitHub ya autenticado en VS Code

Git es un sistema de control de versiones que trabaja con dos repositorios, uno local y privado y uno remoto y compartido. El comando `commit` envía los cambios en las fuentes al repositorio local, mientras que los comandos `pull` y `push` trabajan contra el repositorio remoto y compartido (en este caso alojado en github.com). `pull` (atraer) sincroniza nuestro repo local con lo que hay en remoto trayendo el código actualizado, y `push` (empujar) hace lo contrario, actualiza el contenido del repo remoto llevando lo que hay en local.

Commit y push

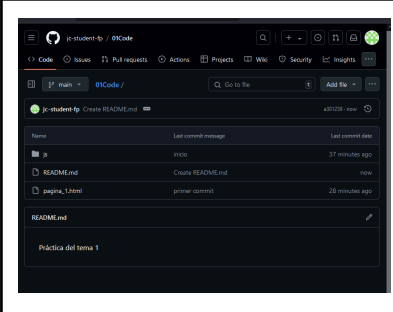
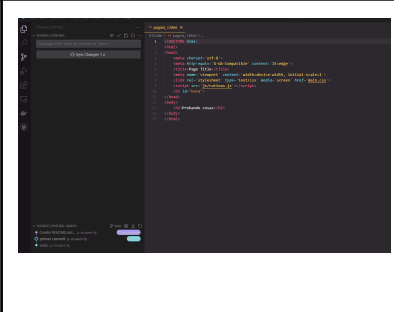
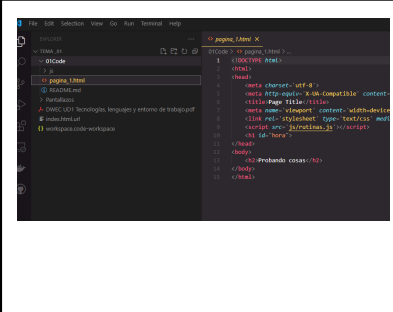
Hay diversas maneras de hacer la tarea propuesta, en mi caso voy a proceder usando exclusivamente VS Code. Muestro los pasos que creo más relevantes de este proceso en los salvados de pantalla que siguen.

 The folder currently open doesn't have a Git repository. You can initialize a repository which will enable source control features powered by Git. Initialize Repository To learn more about how to use Git and source control in VS Code read our docs. You can directly publish this folder to a GitHub repository. Once published, you'll have access to source control features powered by Git and GitHub. Publish to GitHub	 Visual Studio Code interface showing the Source Control panel. The 'Publish Branch' button is visible under the 'SOURCE CONTROL' section. The file explorer on the right shows 'pagina_1.html' and 'JS rutinas.js'.
Comienzo pidiéndole a VS Code en la pestaña de Source Control que inicialice un repositorio Git.	Una vez realizado este primer <code>commit</code> me da la opción de publicar (<code>push</code>)

 Visual Studio Code dialog box: "There are no staged changes to commit. Would you like to stage all your changes and commit?" with "Yes" and "Always" buttons.	 Visual Studio Code interface showing the Source Control panel. The 'Sync Changes' button is visible under the 'SOURCE CONTROL' section.	 GitHub repository page showing the commit history. The commit message "jc-student-tp: primer commit" is visible.
VS Code me avisa sobre el estado staged de los ficheros involucrados, acepto que sean incluidos.	Envío (<code>push</code>)	Y compruebo en GitHub que en efecto todo se ha realizado correctamente.

Pull

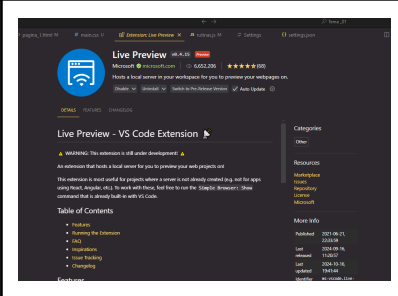
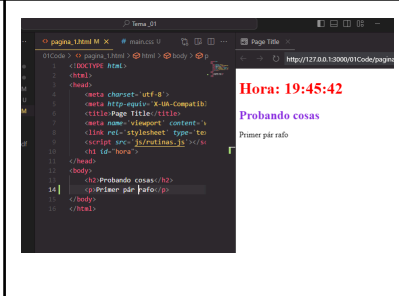
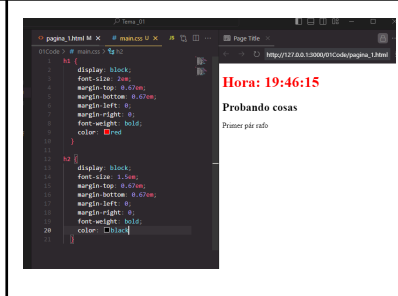
Para mostrar el uso de `pull`, he realizado un `commit` manual en la página de github creando un fichero **readme.md** y luego he sincronizado los cambios.

		
En github, creación de un nuevo fichero y ejecución de un commit	En VS Code, pantalla que me muestra que ha habido cambios en el repo remoto y me da la opción de hacer el pull	pull realizado con éxito y muestra de que el nuevo fichero ya está en el directorio del proyecto en local.

3. Mostrar en tiempo real los cambios en el HTML y CSS a medida que se hacen.

El autoguardado que ofrece VS Code no será suficiente para esto debido a que aunque el fichero haya sido modificado inmediatamente a los cambios, el browser no lo leerá hasta que la página sea refrescada. Por tanto, creo que para hacer esta tarea es necesario buscar y descargar alguna extensión capaz de indicar a un browser que los archivos han cambiado (aunque en realidad no lo hayan hecho, ya que sin autosave no están aún guardados) y los refresque.

De entre las opciones que he encontrado, elijo Live Preview por estar soportada por Microsoft y ser, por tanto, una extensión oficial. Esta extensión monta un servidor y un browser web y es capaz de mostrar los cambios en tiempo real tal como pide el ejercicio, incluso con el autoguardado desactivado.

		
Detalles de la extensión Live Preview	Ejemplo de visualización en tiempo real de cambios en HTML.	Ejemplo de visualización en tiempo real de cambios en CSS.

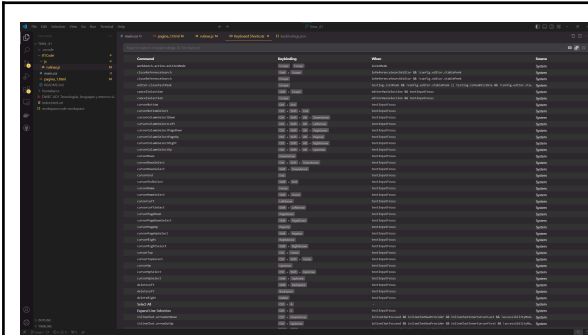
Compruebo además que, ya que monta un servidor, este es accesible desde cualquier otro navegador a través de localhost:3000, por ejemplo un Chrome y que de hecho el resultado de los cambios también se actualiza en tiempo real.


Visualización en Chrome de los cambios en tiempo real (mismo ejemplo de html)

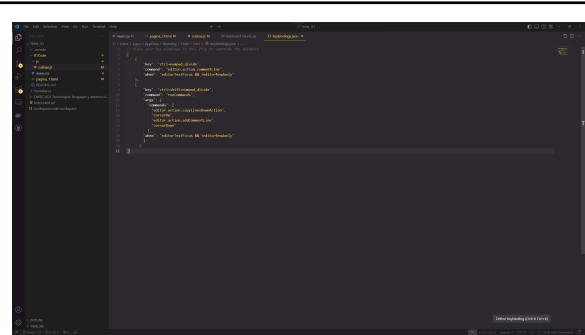
4. Editor de texto con macros.

En **VS Code** se pueden asignar funciones o conjuntos de funciones a una combinación de teclas, quizá el ejemplo más sencillo sea CTRL + s, que realiza un guardado del archivo abierto que tenga el foco en el momento de pulsar esa combinación.

Como ejemplo de macro, me propongo realizar una que situada en una línea de código, la comente y la copie debajo lista para ser editada, una acción que me encuentro haciendo con frecuencia. Para ello debemos acceder a la configuración y de ahí a Atajos de Teclado (Keyboard shortcuts) o pulsando Ctrl+k y Ctrl+s.



Con las keybindings abiertas, solicitamos ver el json (arriba a la derecha) y nos abre el json de configuración de atajos de teclado para el usuario



He añadido dos, una que usa ctrl+/ para comentar y otra que usa ctrl+shift+/ para comentar una línea y copiar la original debajo.

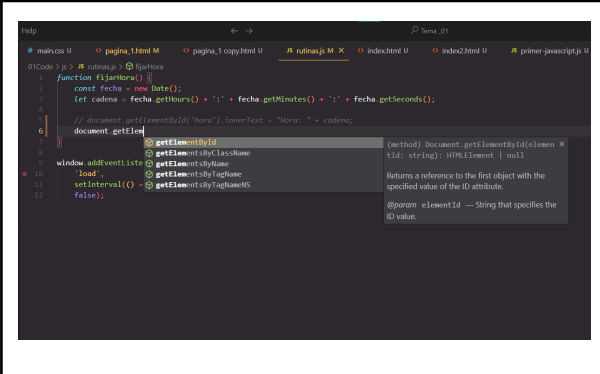
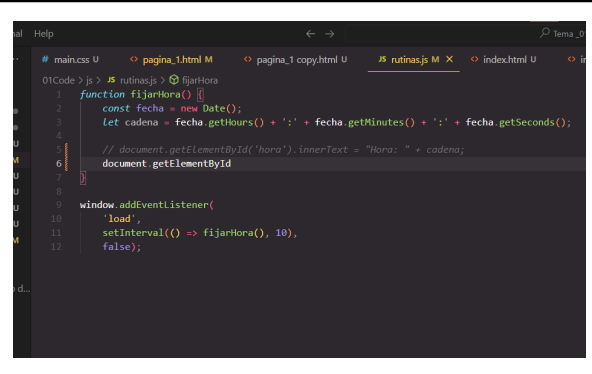
Así queda mi keybindings.json de usuario tras la edición.

```
// Place your key bindings in this file to override the defaults
[
  {
    "key": "ctrl+numpad_divide",
    "command": "editor.action.commentLine",
    "when": "editorTextFocus && !editorReadonly"
  },
  {
    "key": "ctrl+shift+numpad_divide",
    "command": "runCommands",
    "args": {
      "commands": [
        "editor.action.copyLinesDownAction",
        "cursorUp",
        "editor.action.addCommentLine",
        "cursorDown"
      ],
      "when": "editorTextFocus && !editorReadonly"
    }
  }
]
```

Como se puede comprobar, por defecto **VS Code** permite la creación de atajos de teclado para una o varias tareas.

5. Autocompletar con los métodos de las clases del API de JavaScript

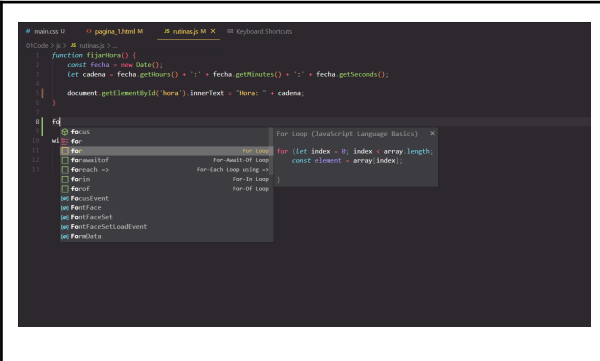
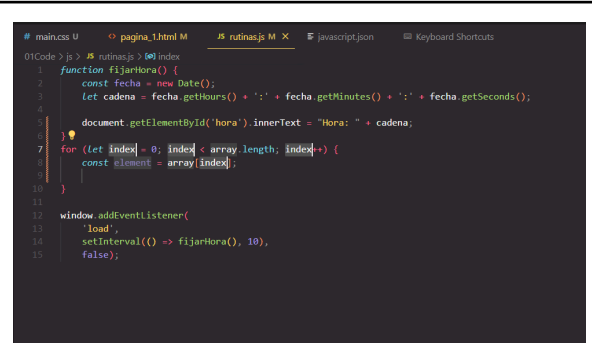
Entre las cosas en las que un IDE debe apoyar al desarrollador está la del autocompletado, el IDE debe poder (o ser configurado para) reconocer el lenguaje utilizado en una expresión y apoyar en el autocompletado de las palabras que se escriben, sean estas reservadas del lenguaje o derivadas de una librería o una API.

	
Vemos como en mitad de la expresión tecleada se nos proponen diferentes métodos para el objeto <code>document</code>, en cualquier momento si hemos eliminado esa ventana la podemos convocar de nuevo pulsando <code>Ctrl + espacio</code>.	Pulsando <code>Tab</code> o <code>Enter</code> nos escribe lo elegido, pudiendo seleccionar la sugerencia tanto con las teclas <code>↑</code> o <code>↓</code>.

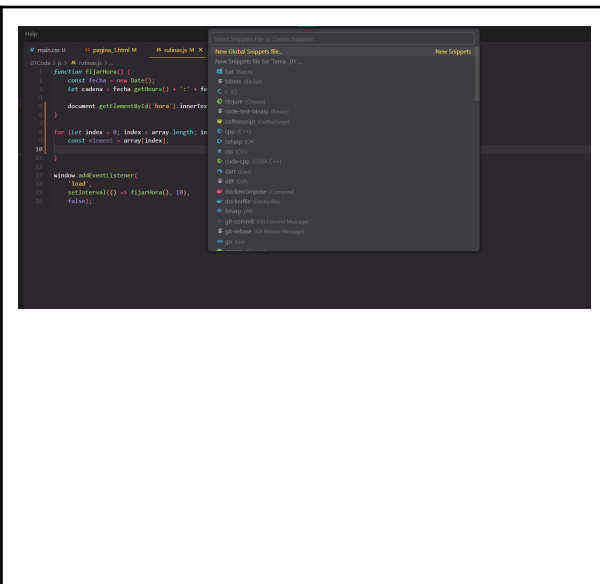
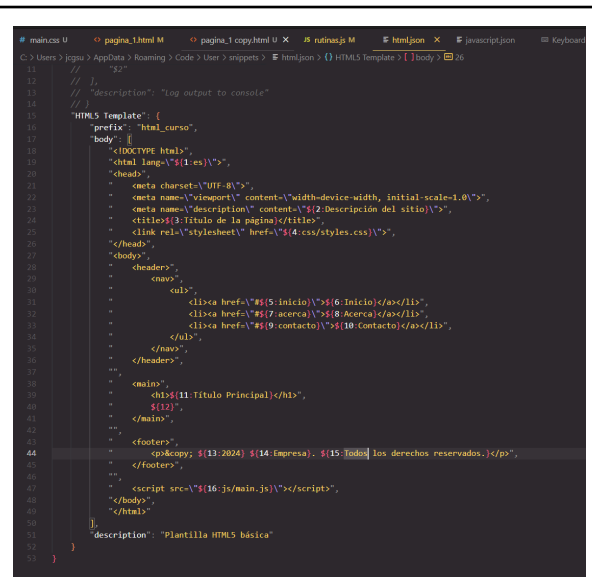
6. Snippets HTML

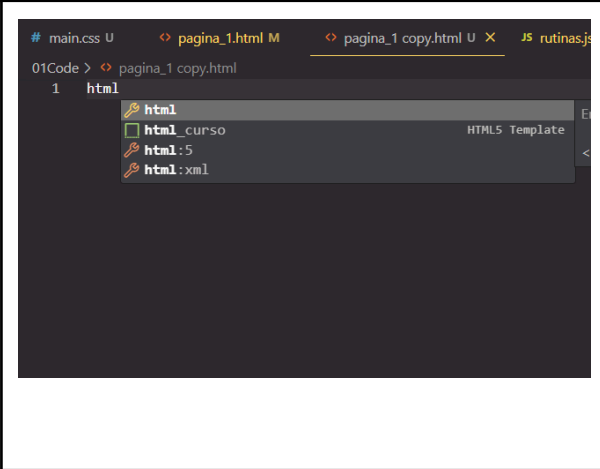
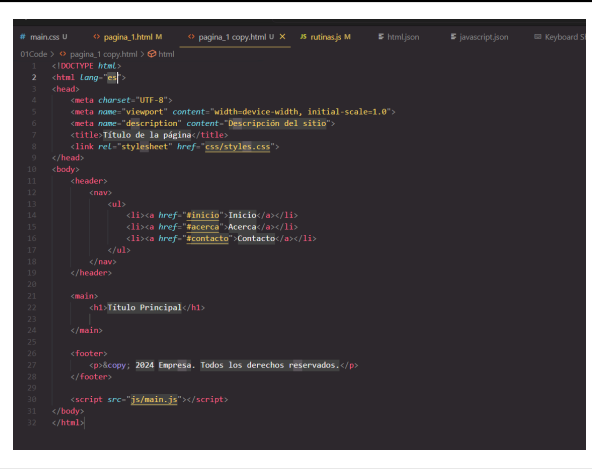
Un Snippet es una plantilla de código que el IDE generará si se lo pedimos para ayudarnos con el desarrollo. Puede ser ejecutable y estar bien formado o ser solo un esquema. Aparece a la vez que los autocompletados y, de haber una coincidencia, se propondrán a la vez que los autocompletados que veíamos en la cuestión anterior. A su vez, es capaz de dejar el cursor (o cursores en caso múltiple)

en donde se le indique. Además, el cursor a través de la tecla Tab podrá ir pasando el foco de una expresión a la siguiente, permitiendo así cambiar nombre de variables, expresiones, etc., y hacerlo en todas las ocurrencias simultáneamente.

	
En este caso escribimos solo <code>fo</code> y el IDE nos sugiere diferentes objetos, palabras restringidas, métodos o snippets (señalados por un cuadrado)	Habiendo elegido el Snippet <code>for loop</code>, nos escribe ese texto y nos posiciona el puntero múltiple en las ocurrencias de la variable que se evaluará en las iteraciones (<code>index</code>), si pulsamos <code>Tab</code> pasaremos el cursor(es) a la siguiente expresión que nos permite editar.

Como en otras ocasiones, **VS Code** nos permite añadir nuestros propios Snippets a través de un archivo json. Se nos pide un Snippet HTML.

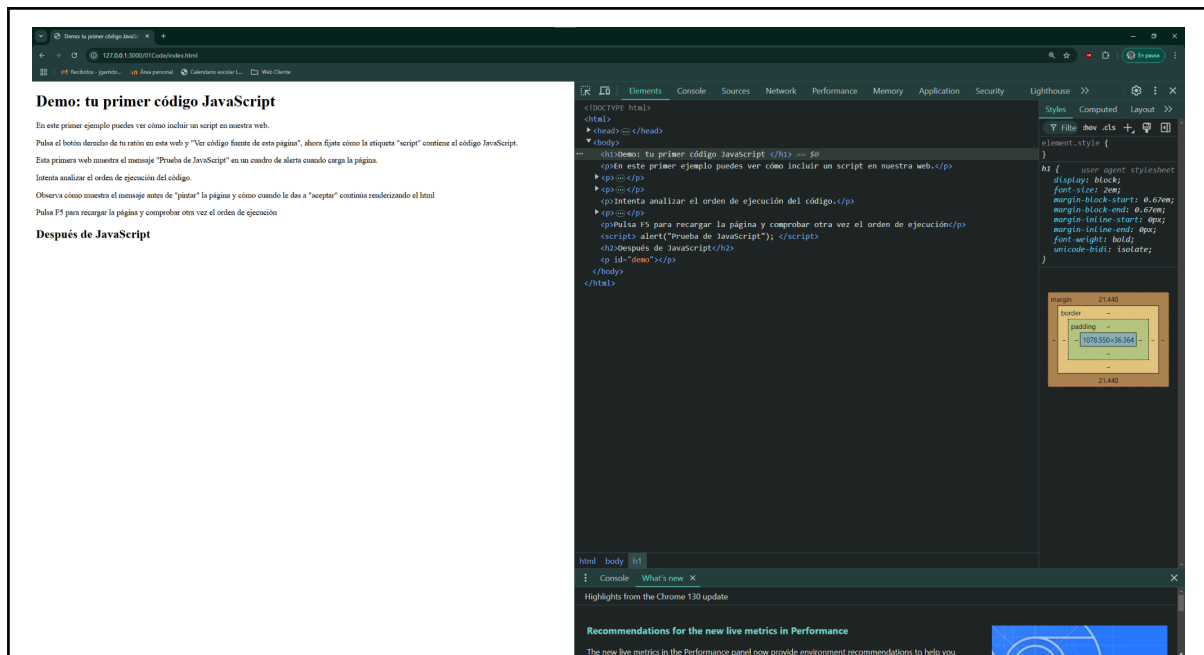
	
En Configuración -> Snippets, encontramos los Snippets clasificados por lenguaje y se nos da la opción de crear uno nuevo global, específico para un lenguaje o para un workspace.	He escrito así el Snippet <code>html_curso</code> que genera una estructura de HTML general con header y footer y que va cambiando el cursor por cada contenido.

	
Aquí vemos como se nos propone este Snippet	Y aquí lo tenemos, una vez solicitado y con el cursor en el primer elemento.

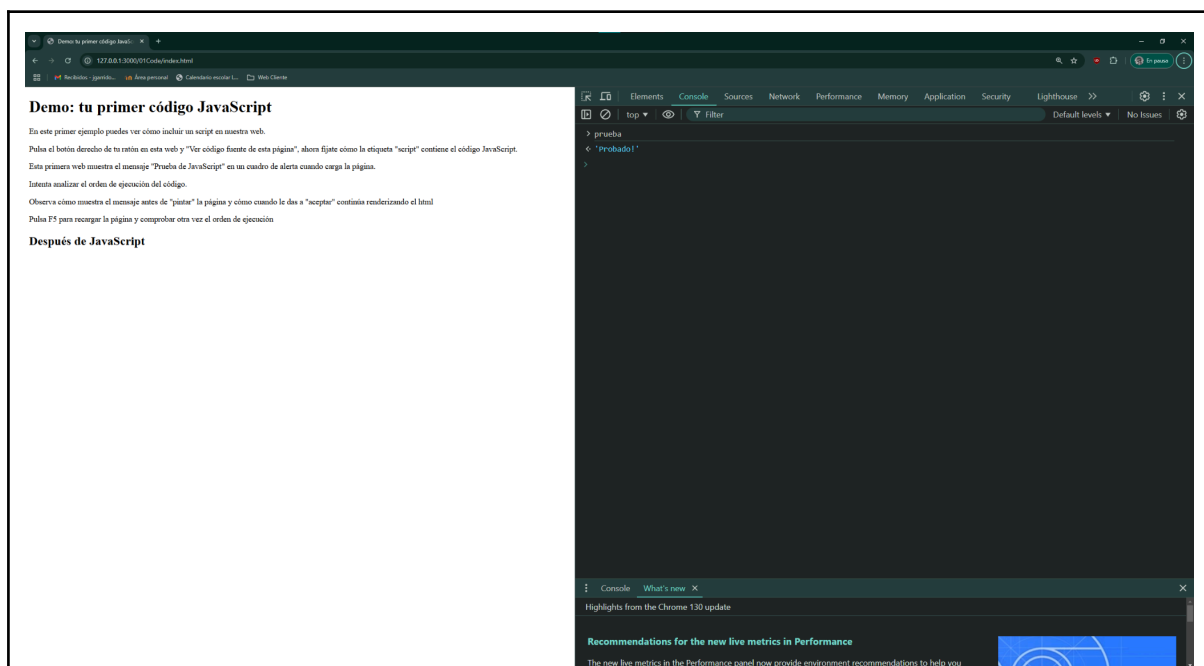
Por otro lado, en **VS Code** disponemos también de la posibilidad de usar abreviaciones Emmet, que son un modo de generar estructuras HTML o CSS, y son muy parecidas a los Snippets excepto porque admiten cierta sintaxis y así, por ejemplo, generar código repetido un cierto número de veces, o con cierta estructura padre>hijo, por ejemplo.

7. Depuración con Chrome (Pestañas, Elements, Console y Sources)

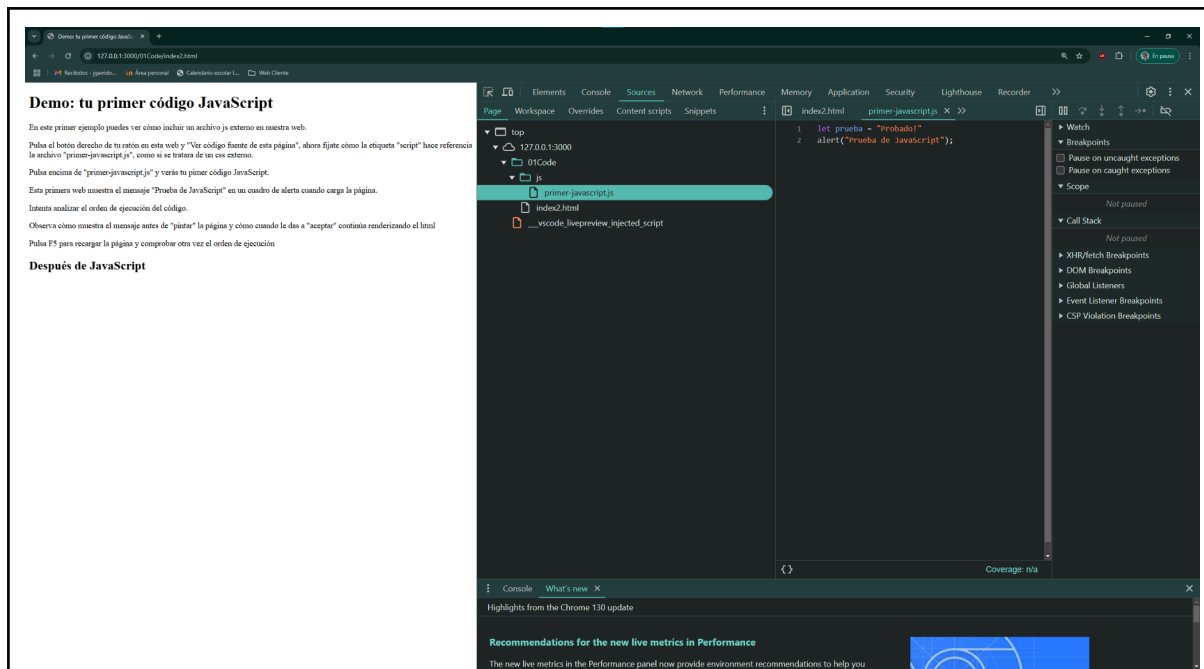
Usando el código de ejemplo que se nos proporcionó, y usando el Live Preview, accedo desde un Chrome a /index.html



Desde un Chrome accedemos a la URL que nos da live preview (no es necesario hacerlo así) y pulsando F12 accedemos a la herramienta para desarrolladores de Chrome. La pestaña seleccionada, Elements, es la pestaña que nos proporciona en tiempo real todos los elementos del documento, es la parte que nos indica lo que el navegador está interpretando en cada instante y en qué resultan las propiedades de cada elemento tras ejecutar todo el código.



La pestaña Console es una herramienta de desarrollo que nos informa de errores en el JavaScript y además nos permite interactuar con el estado del intérprete de JavaScript ejecutando comandos. En este caso he definido una variable (`let prueba = 'Probado!'`) en la página y vemos como puedo conocer su valor desde la consola una vez se ha ejecutado ese script.

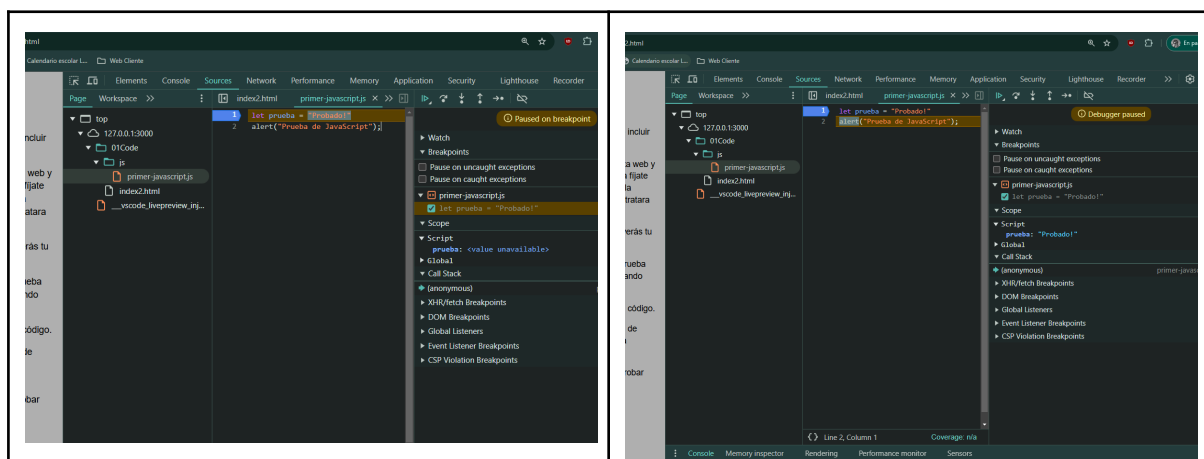


La pestaña Sources se refiere a todos los archivos que ha cargado la página. Se compone de tres secciones:

- Editor: En el centro tenemos un editor de texto capaz de colorear sintácticamente el código y editarlo
- Explorador: En la izquierda tenemos un explorador de todas las fuentes del documento que nos permite seleccionar cada archivo y pasar a editarlo.
- Depurador: En la derecha disponemos de un depurador que nos permite detenernos e ir adelantando en la ejecución, además de evaluar expresiones en cada momento de depuración.

Nota: Vemos en el explorador cómo de hecho ha cargado un script que proviene de la extensión Live Preview y que tiene que ver con las funcionalidades que esta extensión ofrece.

Ejemplo de depuración:



Aquí tenemos detenido el script antes de que se ejecute la declaración de la variable `prueba`. A la derecha podemos ver cómo la variable aún no dispone de un valor definido

Tras dejar que ejecute esa línea (F9), podemos ver cómo ahora esa variable ya tiene un valor asignado.

8. Depuración avanzada en Chrome: Simulación de parámetros típicos de los móviles en Chrome de escritorio: responsive, móvil, geolocalización.

Chrome ofrece en sus herramientas para desarrolladores la posibilidad de desarrollar teniendo en cuenta posibles diferentes entornos al que el desarrollador esté usando, como por ejemplo diferentes resoluciones de pantalla o dispositivos.

Para poder comprobarlo fácilmente, incluyo este código css responsive en la página del ejemplo:

```
<style>
  /* Estilos base para móvil */
  body {
    margin: 0;
    padding: 15px;
    font-family: Arial, sans-serif;
  }

  h1 {
    font-size: 24px;
    color: #35b148;
  }

  p {
    font-size: 16px;
    line-height: 1.4;
  }

  /* Tablet */
  @media (min-width: 768px) {
    body {
      padding: 30px;
      max-width: 700px;
      margin: 0 auto;
    }
  }
```

```

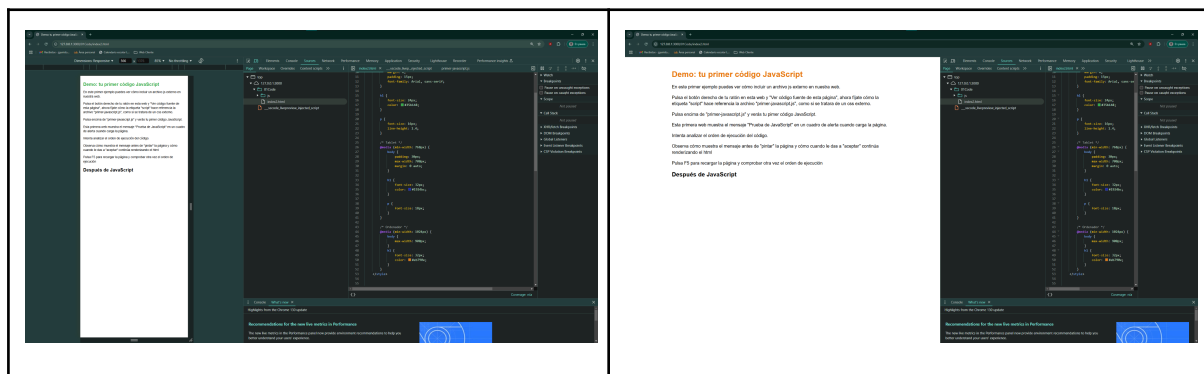
h1 {
    font-size: 32px;
    color: #1924bc;
}

p {
    font-size: 18px;
}
}

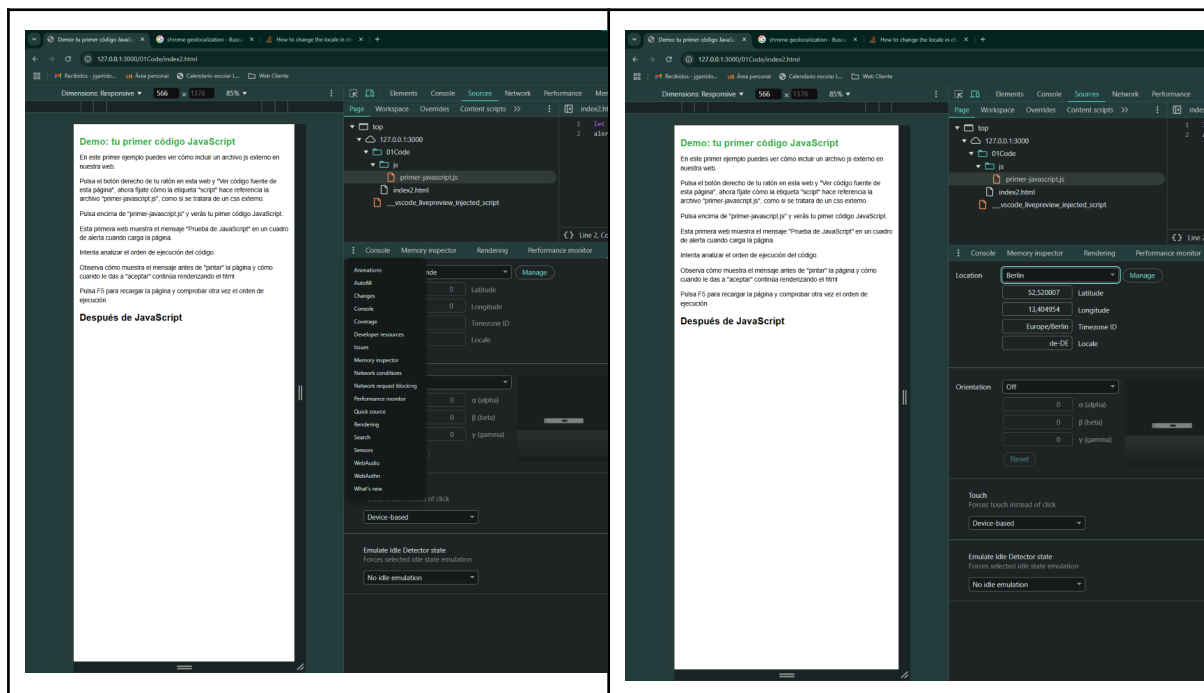
/* Ordenador */
@media (min-width: 1024px) {
    body {
        max-width: 900px;
    }
    h1 {
        font-size: 32px;
        color: #eb790e;
    }
}
</style>

```

Así, el cambio de colores en el H1 permitirá con facilidad ver cómo se están interpretando.



Pulsando en el botón a la izquierda de las pestañas (o Ctrl+shift+m) podemos activar la simulación de dispositivos y resolución del navegador que interpretará nuestra página. Es posible cambiar la resolución de un dispositivo imaginario o simular alguno disponible en el mercado, así como simular una orientación. Aquí podemos comprobar cómo cambia de color el título en función del css que hemos incluido y de la resolución simulada de la pantalla.



Por último, en la sección de herramientas, debajo de la parte dominada por las pestañas que veníamos usando, aparecen un conjunto de útiles que nos pueden proporcionar información al respecto del comportamiento de nuestra página en Chrome.

En particular encontramos una herramienta llamada Sensores en la que podemos establecer un posicionamiento de geolocalización ficticio, una orientación espacial del dispositivo y también un locale diferente para poder observar cómo se comporta nuestra página con diferentes conjuntos de valores.